Assignment 3: MARS Fundamentals

Aryan Dhere

CS 240-02

October 4, 2026

Program 1: Hello World

File: 01_Hello_World.asm

This program loads the print string service into $v0, loads the address of the message into $a0 and uses syscall to print Hello World. It then uses syscall 10 to end the program.

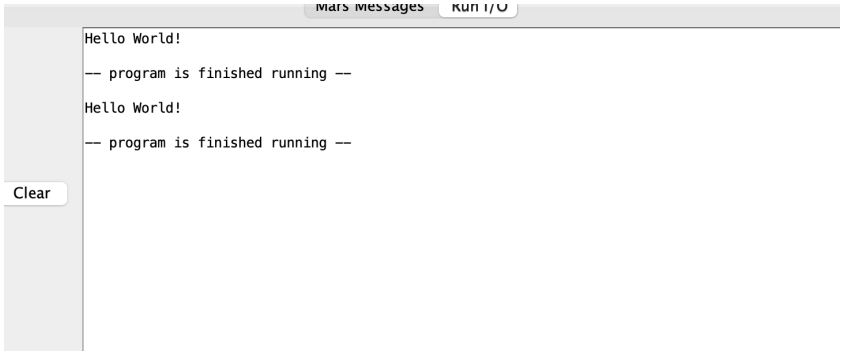


Figure 1. Hello World output.

Register trace

$v0 = 0x00000004: print string service.

$a0 = 0x10010000: address of the message in memory.

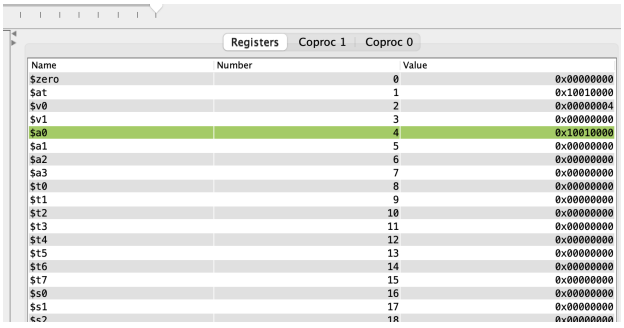


Figure 2. Register values after stepping through the print setup.

Program 2: Print 1 Through 100

File: 02_Print_1_to_100.asm

The counter starts at 1 in $t0. The program prints the counter, adds 1 and repeats the loop until the counter reaches 101. This makes 100 the last number printed.

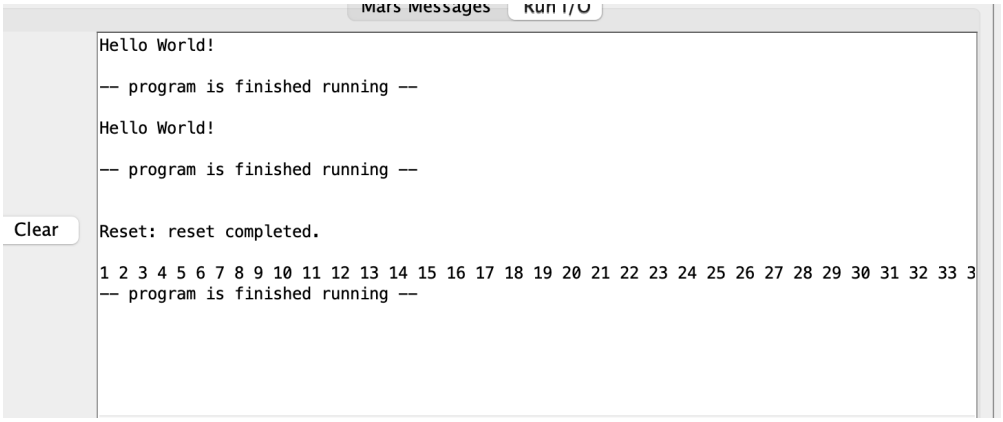


Figure 3. Beginning of the output.

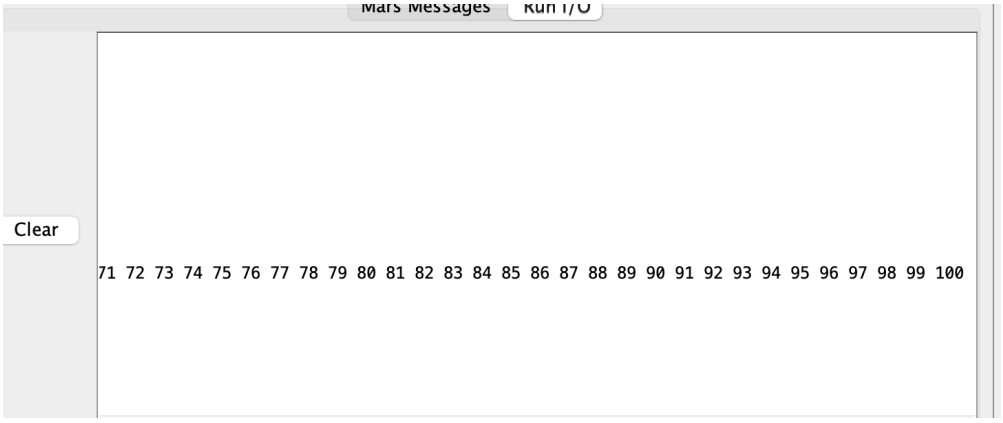


Figure 4. End of the output through 100.

Test result: The first value is 1 and the last value is 100, so both limits of the loop are correct.

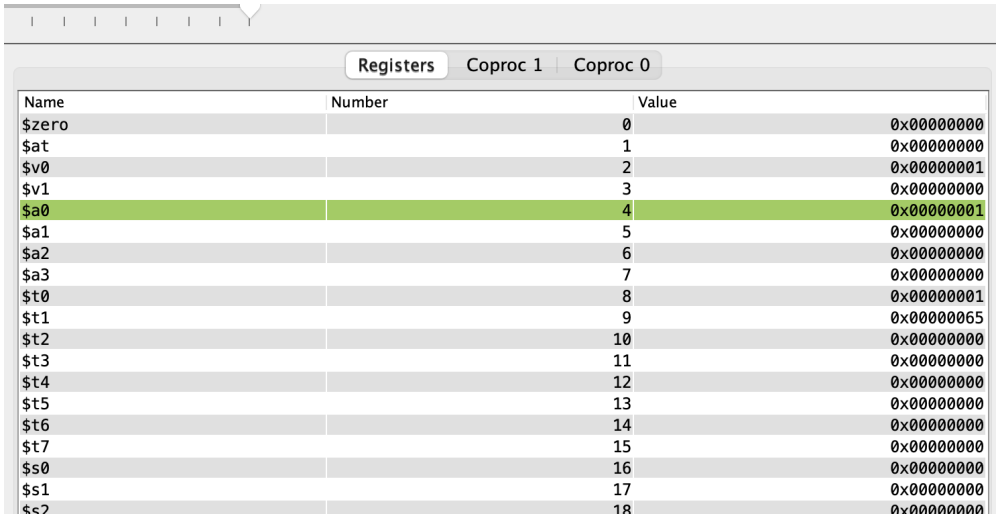
Program 2: Register Trace

$t0 = 0x00000001: current number being printed.

$t1 = 0x00000065: loop stop value, which is 101.

$v0 = 0x00000001: print integer service.

$a0 = 0x00000001: integer sent to the print service.



Name	Number	Value
$zero	0	0x00000000
$at	1	0x00000000
$v0	2	0x00000001
$v1	3	0x00000000
$a0	4	0x00000001
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000001
$t1	9	0x00000065
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000

Figure 5. Register values during the first loop.

Program 3: Sum of Even Integers

File: 03_Sum_Even_Integers.asm

The program starts at 2 and adds each even number to $t2. It increases $t0 by 2 each time and stops when $t0 reaches 102. The final answer is 2550.

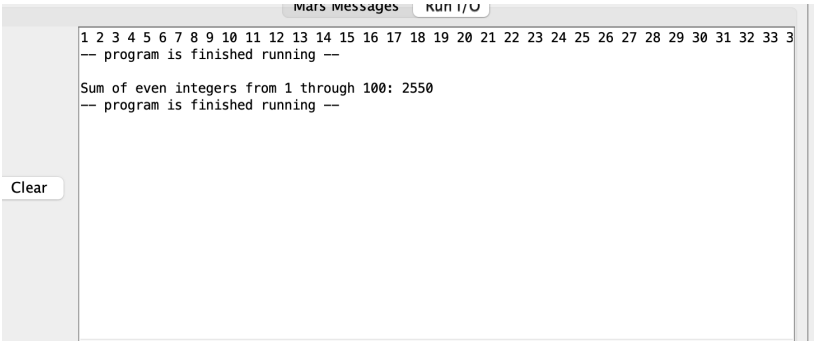


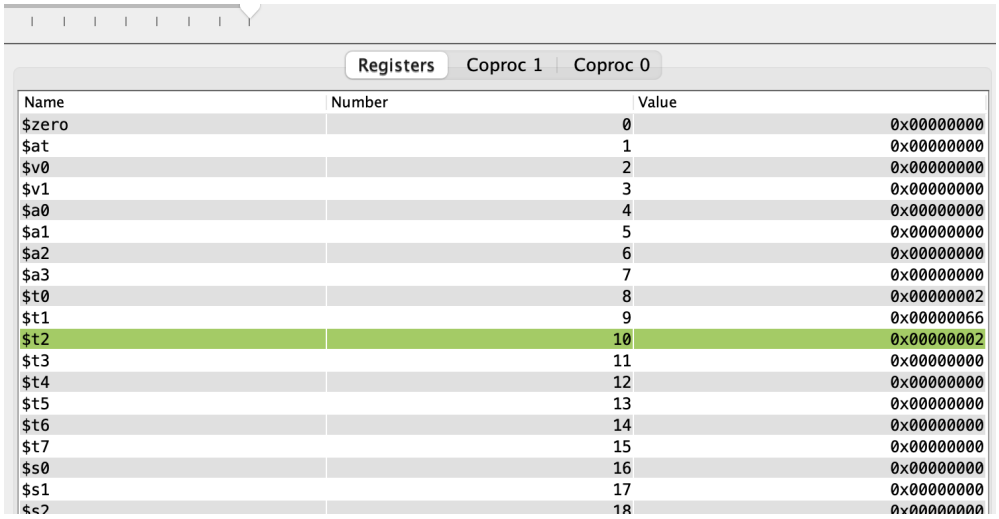
Figure 6. Output showing the sum 2550.

Program 3: Register Trace

$t0 = 0x00000002: current even number.

$t1 = 0x00000066: loop stop value, which is 102.

$t2 = 0x00000002: running sum after the first addition.



Name	Number	Value
$zero	0	0x00000000
$at	1	0x00000000
$v0	2	0x00000000
$v1	3	0x00000000
$a0	4	0x00000000
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000002
$t1	9	0x00000066
$t2	10	0x00000002
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000

Figure 7. Register values during the first calculation.

I also checked the answer using 2 x (1 + 2 + ... + 50). The sum from 1 through 50 is 1275, so 2 x 1275 equals 2550. This matches the program output.

Program 4: Add Values From Memory

File: 04_Add_From_Memory.asm

The program loads 25 and 17 from memory into $t0 and $t1. It adds them in $t2 and stores the answer back into stored_result. The output is 42.

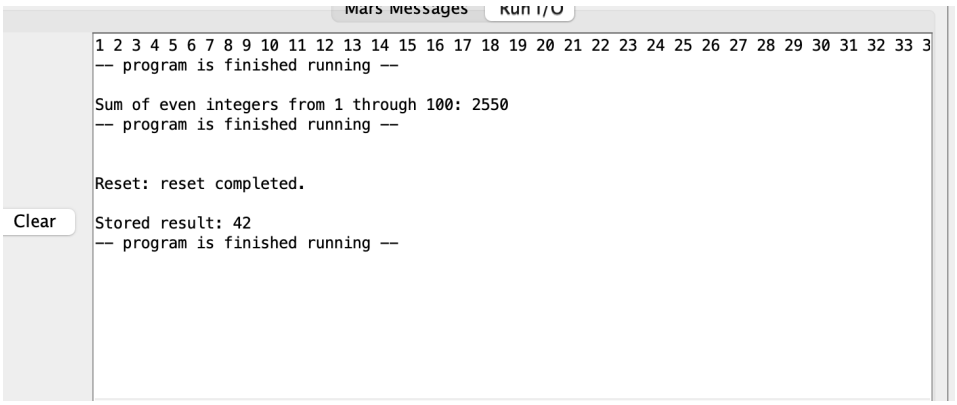


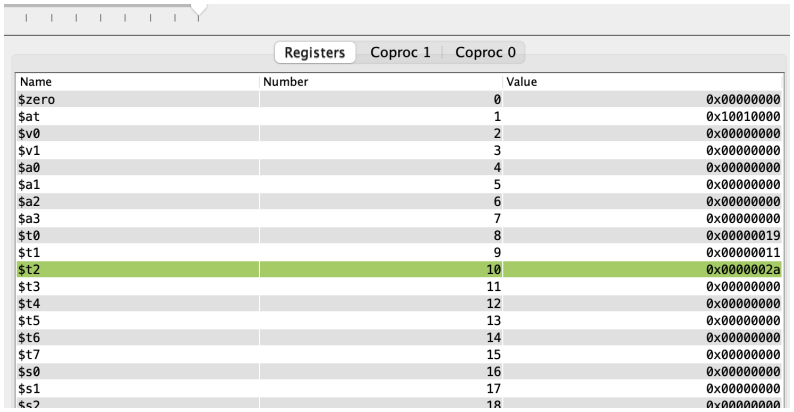
Figure 8. Normal test showing a stored result of 42.

Register trace

$t0 = 0x00000019: 25 loaded from number1.

$t1 = 0x00000011: 17 loaded from number2.

$t2 = 0x0000002A: the answer 42.



Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x00000000
$v1	3	0x00000000
$a0	4	0x00000000
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000019
$t1	9	0x00000011
$t2	10	0x0000002a
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000

Figure 9. Register values after the add instruction.

Program 4: Edge Case

For the edge case, I temporarily changed the values to -25 and 25. The result should be 0 because the values cancel each other out. MARS printed 0, so the edge case passed. I changed the final source file back to 25 and 17 after the test.

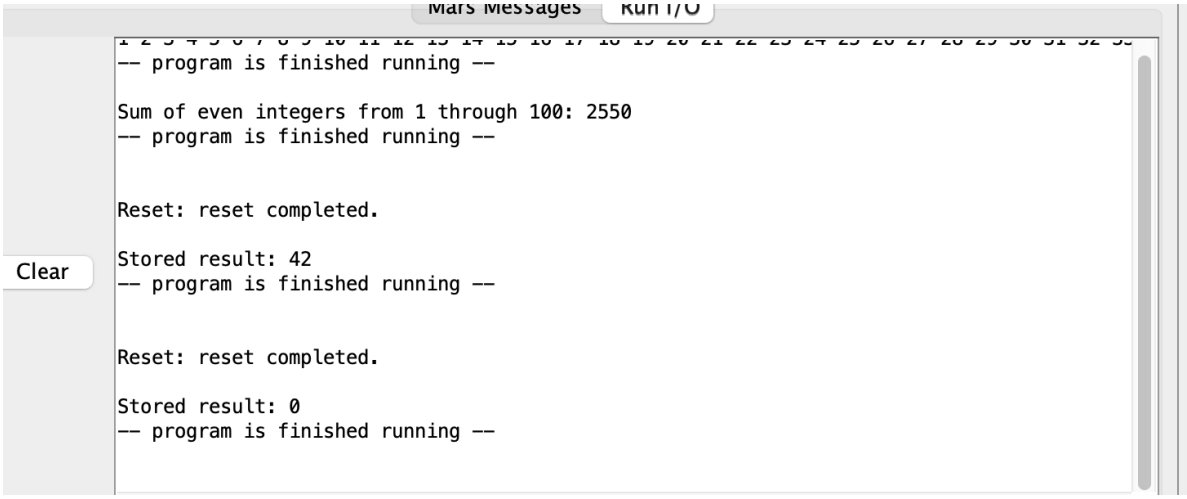


Figure 10. Edge case showing a stored result of 0.

Normal test: 25 + 17 = 42. Result: Pass.

Edge case: -25 + 25 = 0. Result: Pass.

Program 5: FizzBuzz

File: 05_FizzBuzz.asm

The program prints the numbers from 1 through 100. Multiples of 3 print Fizz, multiples of 5 print Buzz and multiples of both print FizzBuzz. The program checks 15 first so a number like 15 does not print only Fizz or only Buzz.



Figure 11. Beginning of the FizzBuzz output.



Figure 12. End of the FizzBuzz output through 100.

Program 5: Register Trace

$t0 = 0x00000001: current number.

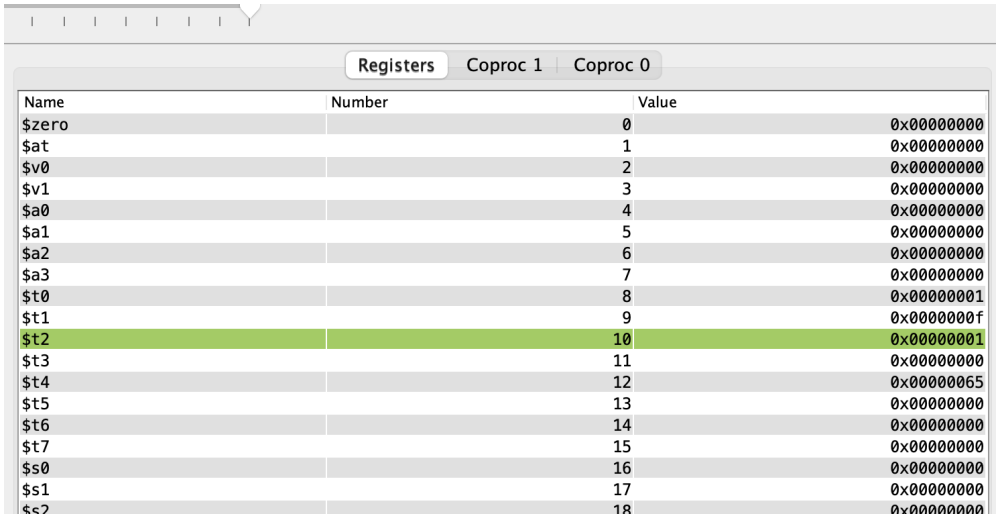
$t1 = 0x0000000F: divisor 15.

$t2 = 0x00000001: remainder copied from HI.

$t4 = 0x00000065: loop stop value 101.

HI = 0x00000001: remainder from 1 divided by 15.

LO = 0x00000000: quotient from 1 divided by 15.



Name	Number	Value
$zero	0	0x00000000
$at	1	0x00000000
$v0	2	0x00000000
$v1	3	0x00000000
$a0	4	0x00000000
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000001
$t1	9	0x0000000f
$t2	10	0x00000001
$t3	11	0x00000000
$t4	12	0x00000065
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000

Figure 13. Register values during the first divisibility check.

I checked the main branches in the output. The value 1 prints as a number, 3 prints Fizz, 5 prints Buzz, 15 prints FizzBuzz and 100 prints Buzz.

Instructions and Pseudo Instructions

MARS uses regular MIPS instructions and pseudo instructions. The pseudo instructions used in these programs are li, la and move. MARS changes them into basic instructions when it assembles the code. Instructions such as add, addi, beq, bne, div, mfhi, j and syscall are basic instructions. The versions of lw and sw that use a label are also convenient forms because MARS resolves the memory address.

Testing Summary

Program 1 printed Hello World and ended normally.

Program 2 started at 1 and ended at 100.

Program 3 produced the correct sum of 2550.

Program 4 passed the normal test and the zero result edge case.

Program 5 printed the correct number, Fizz, Buzz and FizzBuzz cases.

Reflection

This assignment helped me understand what happens inside a MIPS program instead of only looking at the final output. Using the Step button was useful because I could see the values change in the registers during each loop. FizzBuzz was the hardest part because the program had to check 15 before checking 3 or 5. I also learned that li, la and move are pseudo instructions. The memory program showed me how lw and sw move values between memory and registers.

Sources and Collaboration

I used ZyBooks Chapter 3 and the Assignment 3 instructions. I also used ChatGPT for help with the MARS setup, troubleshooting and reviewing the programs. I typed, ran and tested each program in MARS.